

Pemrograman Berorientasi Objek



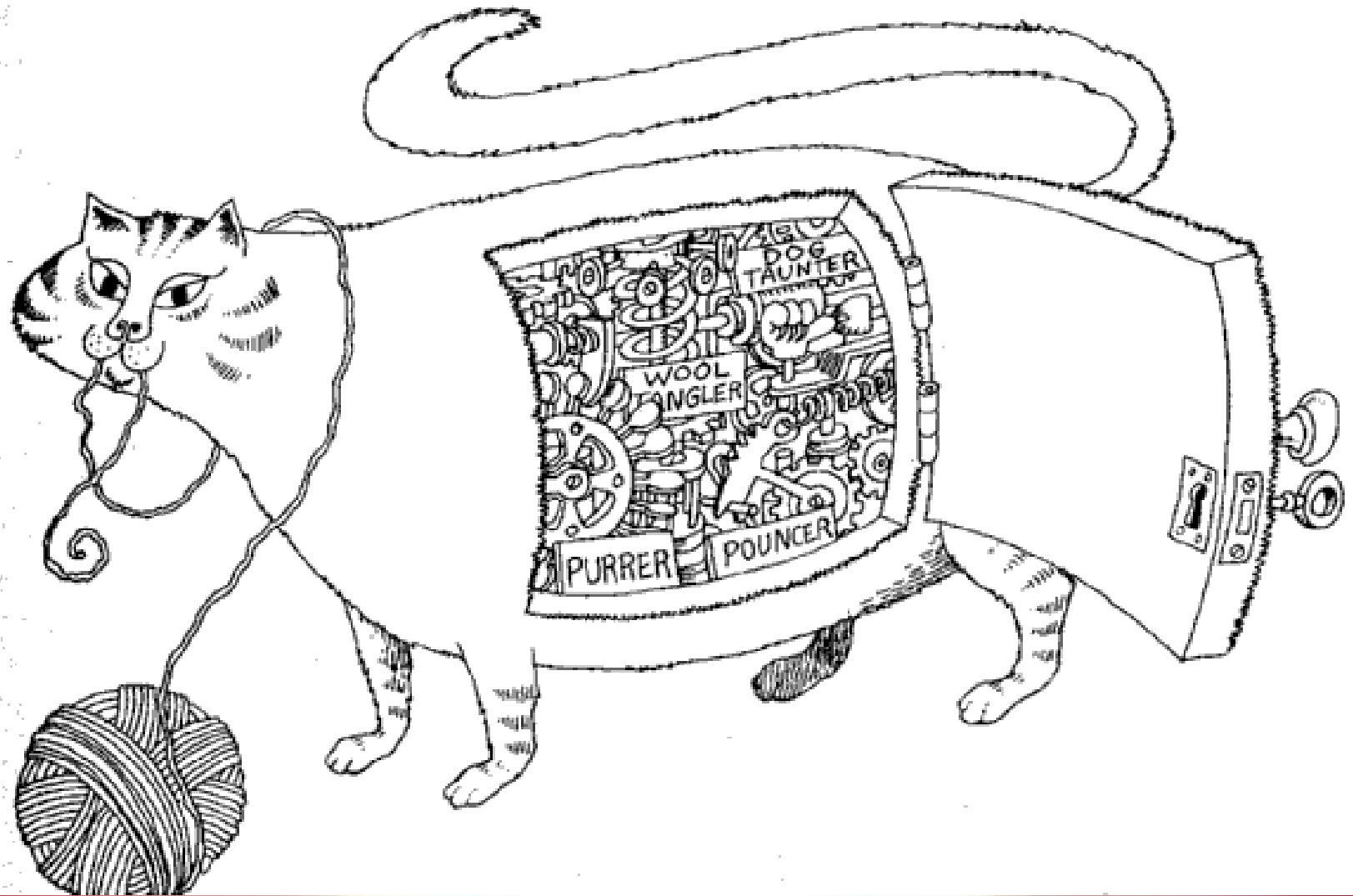
Encapsulation

Encapsulation

- ▶ Encapsulation Concept
- ▶ Access Modifier
- ▶ Constructor
- ▶ Modularity & Package

Encapsulation

- ▶ **Also known as Data Hiding**: Objects have the ability to hide certain parts of themselves from another objects that are not of the same class.



Encapsulation, why?

➤ Modularity

- The source code for an object can be **written and maintained independently** of the source code for the other objects

➤ Data/Information Hiding

- **An object has public interface** that other objects can use to communicate with it.
- The object can maintain **private information and method** that can be changed at any time without affecting other objects that depend on it.

In other words...

- ▶ Technique of making the **fields** in a class cannot be accessed and updated directly, and providing access and editing to the field via **methods**.
- ▶ If a field cannot be accessed and updated by anyone outside the class → **data hiding**
- ▶ **Prevents** the code and data being **randomly accessed** by other code outside the class

Access modifier on class member (attribute & method)

- **private**
- **default**
- **protected**
- **public**

Modifier	Same class	Same Package	Subclass	Any class
private	Y			
default	Y	Y		
protected	Y	Y	Y	
public	Y	Y	Y	Y

private

- ▶ Fields, methods or constructors declared private are **strictly controlled**, which means they **cannot be accessed by anywhere outside the enclosing class**.
- ▶ A standard design strategy is to make **all fields private** and provide **public getter & setter methods** for them.

public

- ▶ **Fields, methods and constructors** declared public (least restrictive) within a public class are **visible to any class** in the Java program, whether these classes are in the same package or in another package.

protected

- **Fields, methods and constructors** declared protected in a superclass can be **accessed only by subclasses** in **other/same packages**.
- Classes in the **same package** can also access protected fields and methods as well, even if they are not a subclass of the protected member's class.

default

- ▶ **no access modifier** is present.
- ▶ Any class, field, method or constructor that has no declared access modifier is **accessible only by classes in the same package.**

```
public class Pizza {
    private int topping; /*member field */
    private String sauce;
    public int getTopping(String user){ //getter
        if (user.equals("Andi")){
            return topping;
        }
        else return 999;
    }
    public void setTopping(int howMany) { //setter
        if(howMany < 3 ){
            this.topping = howMany;
        }
    }
    public String getSauce() { //getter
        return sauce;
    }
    public void setSauce(String someSauce) { //setter
        this.sauce = someSauce;
    }
}
```

```
/* File name : RunEncap.java */
public class RunEncap{

    public static void main(String args[] ){
        Pizza p = new Pizza();
        p.setTopping(2);
        System.out.println("The topping on p is="+p.getTopping("Alex"));
        Pizza q = new Pizza();
        p.setTopping(4);
        System.out.println("The topping on q is="+q.getTopping("Andi"));
    }
}
```

Another use of getter and setter:

- ▶ We can add validation code before accessing or updating the field

```
public int getTopping(String user){ //getter
    if (user.equals("Andi")){
        return topping;
    }
    else return 999;
}
```

```
public void setTopping(int howMany) { //setter
    if(howMany < 3 ){
        this.topping = howMany;
    }
}
```

Access modifier

In more clear
explanations....

Access Modifiers

- *Access Modifiers* specify what classes “see” or “know about”
 - methods or instance variables of other classes
- Effects of visibility ... you **cannot**:
 - reference or instantiate classes which are “invisible” to you
 - call methods that are invisible to you
 - directly access instance variables which are invisible to you
- but **accessors** (getter) and **mutators** (setter) permit safe access



- Generally do not declare classes **private**
- **private** methods:
 - invisible to all other classes
 - **never** inherited by subclasses
 - called *implementation* or *helper* methods because they are written for convenience and code reuse, but are never used outside of the class
- **private** instance variables:
 - same visibility as **private** methods
 - generally, make every instance variable **private**
 - if other classes need to know about a class's instance variables, use accessor and mutator methods (which are **public**)
 - **private** instance variables are pseudo-inherited – the subclass inherits them but can not access them *directly*
 - subclass benefits by using superclass's methods that *do* have access to all instance variables declared at that level in the hierarchy
 - super class can provide accessors/mutators

- The **public** modifier means things are visible to all other classes
- **public** classes:
 - visible to other classes in same package and visible to classes in other packages
 - good for reusing existing code
 - public classes generally go in their own file and *filename must be the same* as **public** class name.
 - generally, *make every class public*
 - exception is **implementation or helper classes** which you would never want other packages to know about and would never want to reuse - these are internal. We will see some of this later.
- **public** methods:
 - visible to all classes that can see the class
 - inherited by subclasses
 - generally *make every method public*
 - exception is **implementation or helper methods** which you are writing for convenience and code reuse - these are for “internal use only”

- **protected** classes:
 - classes *cannot* be declared protected
- **protected** methods:
 - are strictly visible to:
 - subclasses in this or other packages
 - other classes in same package
 - invisible to other packages (except subclasses)
- **protected** instance variables:
 - same visibility as protected methods
 - protected instance variables are visible to:
 - all subclasses (regardless of package)
 - all classes in same package
 - **avoid** using protected instance variables, *except* when you want to give subclasses direct access
- Generally preferable to use accessors and mutators (with **private** instance variables) than **protected** instance variables visible to the entire package

CONSTRUCTOR

Constructor

- ▶ Method that have the **same name as the enclosing class** and **no return type** specifier; **public**
- ▶ The formal argument list of the constructor must match the actual argument list used in any `new` expression.

Example

```
Employee e = new Employee("Jim");
```

you must specify a constructor in Employee:

```
class Employee {  
    String name = null;  
  
    public Employee(String n) {  
        name = n;  
    }  
    ...  
}
```

Default Constructor

- ▶ If a class **does not define a constructor** the compiler will provide a **default constructor with no argument**

Default Constructor

```
public class Building {  
    private String address;  
    public void setAddress(String address) {  
        this.address = address;  
    }  
    public String getAddress() {  
        return this.address;  
    }  
}
```

.....

```
Building x = new Building() ;
```

.....

Default Constructor

- ▶ If the class is **given new constructor**, the **default constructor can't be used**.

Constructor Overloading

- In Java, methods can be **overloaded**. **Method overloading is two or more methods that have the same name but have different parameters**
- We can make **more than one** constructor by **overloaded** it with **another constructor with different parameter**
- In most situations, you will want to **generate objects of a class from different sets of initial defining data.**

Constructor Overloading

```
public class MyClass{  
    int x, y;  
    MyClass() {  
        System.out.println("Inside MyClass()");  
        x=0;  
        y=0;  
    }  
    MyClass(int i, int j) {  
        System.out.println("Inside MyClass(int)");  
        x=i;  
        y=j;  
    }  
}
```

Constructor Overloading

```
MyClass(MyClass obj) {  
    System.out.println("Inside MyClass(MyClass)");  
    x=obj.x;  
    y=obj.y;  
}  
  
void getXValues() {  
    System.out.println("The value are "+x+", "+y);  
}  
}
```

Modularity



Modularity

- ▶ Aspect of syntactically grouping related declarations. (E.g., fields and methods of a data type.)
 - Module in Modula-2, Class/file in C++.
 - **Package**/class in Java.
- ▶ In OOPs, a **class** serves as the basic unit for decomposition and modification. It can be *separately compiled*.

Package

- ▶ Packages are used in Java in-order to **prevent naming conflicts**, to **control access**, to **make searching/locating** and usage of classes, interfaces, enumerations and annotations easier etc.
- ▶ A Package can be defined as a **grouping of related types(classes, interfaces, enumerations and annotations)** providing access protection and name space management.

Package

- ▶ The **package** statement should be the **first line in the source file**. There can be only one package statement in each source file, and it applies to all types in the file.
- ▶ If a package statement is not used then the class, interfaces, enumerations, and annotation types will be put into an **unnamed package**.

default modularity

```
package satu;
```

```
class Animal {}
```

```
package dua;
```

```
import satu.Animal;
```

```
class Dog extends Animal {}
```

→ Will generate error when Dog class is compiled, because Animal class has `default` access and located at different package

public modularity

```
package satu;
```

```
public class Animal {}
```

→Animal class can be instantiated or extended by another class in different package

Another Example

```
// Define a new package called LinkedList
package LinkedList;

public class List {...}
    // class is visible outside of package

class ListElement {...}
    // class is not visible outside of package
```

```
// bring in all classes from LinkedList.
import LinkedList.*;
```

```
// define class A in the default, unnamed package
public class A {...}
```

- ▶ Class List → visible to classes outside package LinkedList
- ▶ Class A may not access ListElement because class A in **different package**
- ▶ Class A may access Class List from different package by **import** the class inside the LinkedList package

Directory Structure of Package

- ▶ The name of the package becomes a **part of the name of the class**
- ▶ The **name of the package** must match the **directory structure** where the corresponding bytecode resides.

`LinkedList.List`

must exist in file

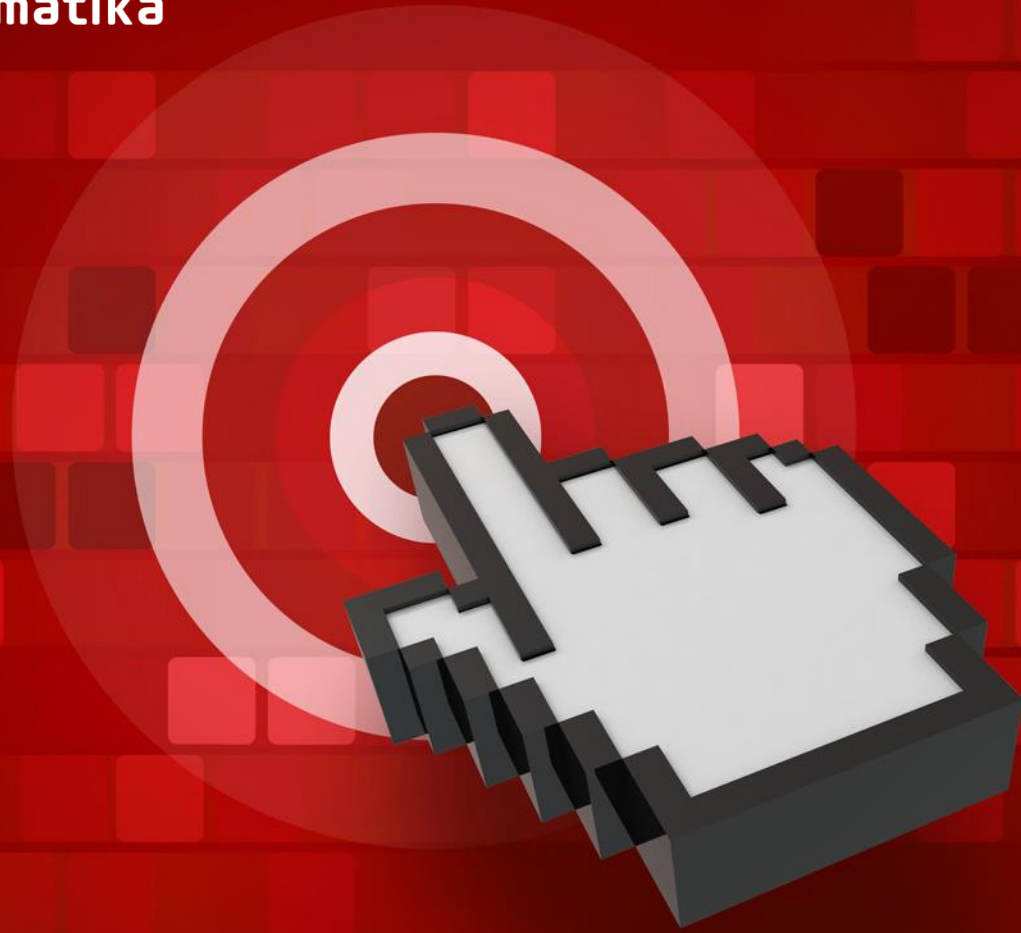
`LinkedList/List.class`

Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU